PROGRAMMING IN JAVA

ASSIGNMENT 5 – CO5

DATA PERSISTENCE ENGINEERING – FILES AND DATABASES

Course Outcome – CO5: Java I/O – byte, character and buffered streams, the File class, serialization; JDBC – architecture and drivers, Statement, PreparedStatement, CallableStatement, transactions, batch processing and metadata (Sessions 39–48)

Nature: Individual

Duration: One Week

Total Marks: 50

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task is stated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment5** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assessment Components

PART A – Persistence Design & Pseudocode: 25 Marks

PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Almost every real application must store data so that it survives after the program ends. Java offers several ways to do this – text files, binary streams, object serialization and relational databases through JDBC.

This assignment investigates a fundamental question:

> **How should a Java engineer decide where and how data is stored, and how can the program be made correct, safe and efficient as the data grows?**

You are expected to demonstrate the following progression:

> **Model → Choose a Store → Design → Protect → Implement → Measure → Compare → Decide**

Simply obtaining a program that saves and loads data is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

PART A – PERSISTENCE DESIGN & PSEUDOCODE **25 Marks**

All pseudocode must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode.

TASK A1 – DESIGN A FILE FORMAT **7 Marks**

Bloom's Level: K4 – Analyse

Transport Office Bus-Pass Records

The university transport office issues bus passes. Each pass record contains: pass ID, student register number, name, route number, boarding stop, validity date and amount paid. About 8,000 passes are issued each year. The office wants the records stored in a file and needs to (i) add a new pass, (ii) find a pass by pass ID, and (iii) list all passes on a given route.

A1.1 – Represent the Record

The university transport office maintains approximately 8,000 bus-pass records every year. Each record contains Pass ID, Register Number, Name, Route Number, Boarding Stop, Validity Date and Amount Paid.

1. Delimited text file format

A CSV file can be used to store bus-pass information.

Example:

P1001,192311001,Arun,12,Central Stop,2026-12-31,4500

P1002,192311002,Divya,7,College Gate,2026-12-31,3800

P1003,192311003,Ravi,5,Bus Stand,2026-12-31,4200

Field	**Data type**	**Example**
Pass ID	String	P1001
Register Number	String	192311001
Name	String	Arun
Route Number	Integer	12
Boarding Stop	String	Central Stop
Validity Date	Date	2026-12-31
Amount Paid	Double	4500.00

How to handle a name containing a comma:

- Place the field inside double quotes.
- Example: "Arun, Kumar".
- While reading, use a CSV parser or a suitable quoted-field parsing method instead of simply splitting at every comma.

2. Binary file format using DataOutputStream

Binary records can be stored using DataOutputStream.

Field order:

1. Pass ID – writeUTF()
2. Register Number – writeUTF()
3. Name – writeUTF()
4. Route Number – writeInt()
5. Boarding Stop – writeUTF()
6. Validity Date – writeUTF()
7. Amount Paid – writeDouble()

Example Java statements:

```
DataOutputStream out = new DataOutputStream(
    new FileOutputStream("passes.dat"));

out.writeUTF("P1001");
out.writeUTF("192311001");
out.writeUTF("Arun");
out.writeInt(12);
out.writeUTF("Central Stop");
out.writeUTF("2026-12-31");
```

```
out.writeDouble(4500.0);


out.close();
```

A1.2 – Design the Operations

Operation 1: Add a new bus pass

Algorithm:

```
BEGIN
   INPUT passID, regNo, name
   INPUT routeNo, boardingStop
   INPUT validityDate, amount

   OPEN file using BufferedWriter
   APPEND the new pass details
   WRITE the fields separated by commas
   WRITE a new line
   CLOSE the file

   DISPLAY "Bus pass added successfully"
END
```

Operation 2: Find a pass using Pass ID

```
BEGIN
   INPUT searchID
   SET found = false

   OPEN file using BufferedReader
```

```
    WHILE another line is available
        READ one line
        SPLIT the line into fields

        IF the record contains valid fields THEN
            IF Pass ID equals searchID THEN
                DISPLAY the pass details
                SET found = true
                EXIT loop
            END IF
        END IF
    END WHILE

    IF found = false THEN
        DISPLAY "Pass not found"
    END IF

    CLOSE the file
END
```

Operation 3: List all passes on a given route

```
BEGIN
    INPUT requiredRoute
    SET found = false
```

```
    OPEN file using BufferedReader

    WHILE another line is available
        READ one line
        SPLIT the line into fields

        IF the record is valid THEN
            IF route number equals requiredRoute THEN
                DISPLAY the pass details
                SET found = true
            END IF
        END IF
    END WHILE

    IF found = false THEN
        DISPLAY "No passes found for this route"
    END IF

    CLOSE the file
END
```

Stream classes and their uses

Stream class	**Purpose**
FileReader	Reads character data from a file.
FileWriter	Writes character data to a file.

BufferedReader	Reads text efficiently using a buffer.
BufferedWriter	Writes text efficiently using a buffer.
FileInputStream	Reads binary data from a file.
FileOutputStream	Writes binary data to a file.

A1.3 – Design for Damaged Data

1. Damaged line

- Check whether every record contains the required number of fields.
- Reject or skip records with missing or extra fields.
- Store invalid records in a separate error file.
- Display a message identifying the damaged record.

2. Missing file

- Check whether the file exists before reading.
- If the file does not exist, display an error message.
- Use exception handling with FileNotFoundException.
- Create a new file when adding the first record, if appropriate.

3. Incomplete write due to power failure

- Write new records into a temporary file first.
- Flush buffered data and close the file properly.
- Replace the original file only after the temporary file is completely written.
- Maintain a backup to recover previous records.

Alternative Concept – RandomAccessFile

If fixed-length records are used with RandomAccessFile, the program can directly access a record by its position.

Formula:

$$\text{Position} = i \times \text{recordSize}$$

Example:

```
RandomAccessFile file =
    new RandomAccessFile("passes.dat", "r");


int recordSize = 100;
int recordNumber = 5;


file.seek((long) recordNumber * recordSize);
```

Complexity comparison

Operation	**Text file**	**Fixed-length RandomAccessFile**
Find by ID	O(n)	O(1) by record position
Add record	O(1) append	O(1) at end
List route records	O(n)	O(n)
Storage	Variable length	Fixed length

RandomAccessFile is faster when the record position is already known. However, fixed-length fields can waste space because unused characters must be padded.

TASK A2 – DON'T READ WHAT YOU DON'T NEED **5 Marks**

Bloom's Level: K4 – Analyse

A program reads a 50 MB attendance log one byte at a time using FileInputStream.read() to count the number of lines.

A2.1 – Predict the Cost

A program reads a 50 MB attendance log one byte at a time using FileInputStream.read().

Calculation:

1 MB = 1,048,576 bytes.

\[50 \times 1,048,576 = 52,428,800\text{ bytes} \]

Therefore, the program makes approximately 52,428,800 read calls when reading one byte per call.

Why is it slow?

- It performs a very large number of read operations.
- Each read operation has method-call overhead.
- Frequent interaction with the operating system increases processing time.
- The program does not efficiently use buffered reading.

A2.2 – Redesign Using Buffering

A BufferedInputStream can read data in blocks instead of one byte at a time.

Java code:

```
import java.io.*;


public class BufferedLineCounter {
   public static void main(String[] args) throws IOException {


      int lines = 0;


      try (BufferedReader br = new BufferedReader(
            new FileReader("attendance.txt"))) {


         String line;


         while ((line = br.readLine()) != null) {
```

```
            lines++;
        }
    }

    System.out.println("Total Lines: " + lines);
  }
}
```

Advantages of buffering:

1. Reads multiple characters into memory at once.
2. Reduces the number of operating-system read operations.
3. Improves file-reading performance.
4. Reduces repeated access to the disk.
5. Makes large-file processing more efficient.

A2.3 – When Buffering Does Not Help

Buffering gives little benefit when:

- The file is very small.
- The data is already available in memory.
- The program performs only one small read operation.
- The program does not repeatedly access the file.

Alternative Concept – Files.readAllLines()

The complete file can be read into memory using Files.readAllLines().

Example:

```
import java.nio.file.*;
import java.io.*;
import java.util.*;
```

```
public class ReadAllLinesDemo {
    public static void main(String[] args) throws IOException {

        List<String> lines =
            Files.readAllLines(Path.of("attendance.txt"));

        System.out.println("Total Lines: " + lines.size());
    }
}
```

Complexity comparison

Method	Time complexity	Space complexity
Buffered reading	O(n)	O(b)
Files.readAllLines()	O(n)	O(n)

Here, n represents the file size and b represents the buffer size.

Buffered reading is suitable for large files because it uses limited memory. Files.readAllLines() is suitable for smaller files that can comfortably fit in available memory.

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN **5 Marks**

Bloom's Level: K4 – Analyse

A student project has three classes – StudentDB, CourseDB and MarksDB. Each one opens a connection, builds an SQL string, executes it, reads the ResultSet into objects and closes the connection. Large parts of the three classes are almost identical.

A3.1 – Identify the Common Pattern

The project contains three classes:

- StudentDB
- CourseDB
- MarksDB

All three classes perform similar database operations.

Common steps:

1. Establish a database connection.
2. Create an SQL query.
3. Execute the query.
4. Read the ResultSet.
5. Convert database records into Java objects.
6. Close the connection.

Differences:

StudentDB	**CourseDB**	**MarksDB**
Accesses student table	Accesses course table	Accesses marks table
Student details	Course details	Marks details
Student ID	Course ID	Marks ID

The repeated code can be reduced by using the Data Access Object (DAO) design pattern.

A3.2 – Design a Generic Data-Access Interface

A generic DAO interface provides common methods for database operations.

DAO interface:

import java.util.List;

```
public interface Dao<T, K> {

    void insert(T object);

    T findById(K id);

    List<T> findAll();

    void update(T object);

    void delete(K id);
}
```

Student class:

```
public class Student {

    int id;
    String name;
    String department;

    public Student(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
}
```

StudentDao implementation:

```
import java.util.*;

public class StudentDao implements Dao<Student, Integer> {

   List<Student> students = new ArrayList<>();

   public void insert(Student s) {
      students.add(s);
   }

   public Student findById(Integer id) {
      for (Student s : students) {
         if (s.id == id)
            return s;
      }
      return null;
   }

   public List<Student> findAll() {
      return students;
   }

   public void update(Student s) {
      for (int i = 0; i < students.size(); i++) {
```

```
            if (students.get(i).id == s.id) {
                students.set(i, s);
                return;
            }
        }
    }

    public void delete(Integer id) {
        students.removeIf(s -> s.id == id);
    }
}
```

Note: This example uses an in-memory list to demonstrate the DAO interface. A complete JDBC implementation would execute SQL queries against the student table.

A3.3 – Map Objects to Tables

The Student class is mapped to a database table named student.

Java field	Database column	Data type
id	student_id	INT
name	student_name	VARCHAR(100)
department	department	VARCHAR(50)

SQL table creation:

```
CREATE TABLE student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    department VARCHAR(50)
);
```

Alternative Concept – SQL inside service or GUI classes

If SQL queries are directly written inside service or GUI classes:

- SQL code becomes repeated.
- Changing a database column requires modifying multiple classes.
- Maintenance becomes difficult.
- Database logic and application logic become tightly connected.

With DAO, SQL changes are mainly handled in the DAO implementation.

Complexity comparison: Both approaches generally have the same time complexity for the same SQL operation. The DAO pattern improves maintainability and code reuse rather than automatically making database queries faster.

TASK A4 – WHAT SHOULD BE SERIALIZED? **4 Marks**

Bloom's Level: K4 – Analyse

A hostel management program saves its in-memory state by serializing a Hostel object, which holds a list of Room objects, each holding a list of Resident objects. Each Resident has a name, register number, password and a reference to a live database Connection.

A4.1 – Decide What to Persist

The Hostel object contains:

- List of Room objects.
- Each Room contains a list of Resident objects.
- Each Resident contains name, register number, password and a database Connection reference.

Fields to serialize:

Field	Serialize?	Reason
Hostel details	Yes	Required for saving hostel information
Room list	Yes	Required to restore room information
Resident name	Yes	Required resident information
Register number	Yes	Identifies each resident
Password	No	Sensitive information should not be stored in plain serialized data
Database Connection	No	Live connection cannot be meaningfully restored from a saved file

The transient keyword prevents a field from being serialized.

Example:

```
import java.io.Serializable;


class Resident implements Serializable {


    String name;

    String registerNumber;


    transient String password;

    transient java.sql.Connection connection;

}
```

Object graph serialization

When the Hostel object is serialized, Java follows its references and serializes the reachable serializable objects, including Room and Resident objects.

The transient fields are not saved.

Source: Assignment 5, pages 3–4.

ASSIGNMENT5.pdf

A4.2 – Plan for Change

Suppose a new field called phoneNumber is added to the Resident class after some files have already been saved.

Example:

```
class Resident implements Serializable {


   private static final long serialVersionUID = 1L;



   String name;

   String registerNumber;

   String phoneNumber;

}
```

What happens when an old file is read?

- If the class uses a compatible serialVersionUID, the old serialized file can generally be read.
- The new phoneNumber field receives its default value, which is null for a String.
- If the serialVersionUID is incompatible, Java may throw InvalidClassException.

Importance of serialVersionUID:

- Identifies the version of a serializable class.
- Helps Java check compatibility between saved objects and current class definitions.
- Supports controlled changes to serialized classes.

Alternative Concept – CSV or JSON

Instead of Java serialization, hostel data can be saved in CSV or JSON format.

Feature	**Java serialization**	**CSV / JSON**
Format	Binary	Text
Readability	Not human-readable	Human-readable
File size	Often larger due to serialization metadata	Depends on the data and formatting
Save/load	Generally efficient for object graphs	Requires parsing and mapping
New field	Version compatibility must be considered	Can often be added with parser/default handling

For n residents, saving and loading requires processing the resident data, generally O(n) time in either approach. Actual file size and performance depend on the object structure and representation.

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY **4 Marks**

Bloom's Level: K4 – Analyse

A5.1 – Transaction Design

A student transfers ₹500 from a hostel wallet to a canteen wallet.

The transfer must be completed as one transaction. If one update fails, the entire transaction should be rolled back.

Pseudocode:

```
BEGIN
  CONNECT TO DATABASE

  DISABLE AUTO COMMIT

  TRY
    UPDATE hostel_wallet
    SUBTRACT 500 FROM BALANCE

    UPDATE canteen_wallet
    ADD 500 TO BALANCE

    IF both updates are successful THEN
      COMMIT TRANSACTION
      DISPLAY "Transfer successful"
    ELSE
```

```
        ROLLBACK TRANSACTION
        DISPLAY "Transfer failed"
    END IF

  CATCH DATABASE EXCEPTION
    ROLLBACK TRANSACTION
    DISPLAY "Transaction failed"

  FINALLY
    CLOSE DATABASE CONNECTION
END
```

Explanation:

- setAutoCommit(false) starts manual transaction control.
- commit() permanently saves both updates.
- rollback() cancels both updates if an error occurs.
- This prevents money from being lost or created when one update fails.

A5.2 – Injection Analysis

The given unsafe login query is:

```
String sql = "SELECT * FROM users WHERE id='" +
             id + "' AND pwd='" + pwd + "'";
```

Problem:

The program directly joins user input with the SQL query. An attacker can enter a crafted value that changes the meaning of the query.

Example input:

ID: admin' OR '1'='1' --

Password: anything

The SQL query can become logically equivalent to:

SELECT * FROM users

WHERE id='admin' OR '1'='1' -- ' AND pwd='anything'

The condition '1'='1' is always true, and -- comments out the remaining SQL text in MySQL. This can allow an invalid login to be accepted, depending on the query and database configuration.

Safe solution using PreparedStatement:

```
import java.sql.*;


public class SafeLogin {


   public static void main(String[] args) throws Exception {


      Connection con = DriverManager.getConnection(

         "jdbc:mysql://localhost:3306/testdb",

         "root",

         "password"

      );


      String sql =

         "SELECT * FROM users WHERE id=? AND pwd=?";


      PreparedStatement ps = con.prepareStatement(sql);
```

```
        ps.setString(1, "admin");
        ps.setString(2, "anything");

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            System.out.println("Login Successful");
        } else {
            System.out.println("Invalid Login");
        }

        rs.close();
        ps.close();
        con.close();
    }
}
```

Advantages of PreparedStatement:

1. Separates SQL instructions from user input.
2. Prevents user input from being treated as SQL commands.
3. Improves query safety.
4. Supports parameterized queries.
5. Makes repeated execution of the same query more efficient in many cases.

Alternative Concept – Two independent auto-commit updates

If the two wallet updates are executed separately with auto-commit enabled, the first update may succeed while the second fails.

For example:

- Hostel wallet: ₹500 deducted successfully.
- Canteen wallet: ₹500 not credited because of an error.
- Result: ₹500 is lost from the combined wallet balances.

Using a single transaction prevents this partial-update problem.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION 25 Marks

Part B assesses whether your design reasoning survives contact with actual computation. You are expected to:

Implement → Instrument → Stress → Measure → Compare → Interpret

Execution screenshots alone are insufficient.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE 7 Marks

Bloom's Level: K4 – Analyse

Implement the bus-pass file store designed in Task A1 (text format) and the buffered line counter from Task A2.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE (7 Marks)

B1.1 – Complete the Code (TODO 1)

The program must read the bus-pass file line by line using BufferedReader. It must skip damaged records that do not contain exactly five fields and return the matching pass.

Complete Java program:

```
import java.io.BufferedReader;

import java.io.BufferedWriter;

import java.io.IOException;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {

    private final Path file;

    PassStore(Path file) {
        this.file = file;
    }

    public void add(String passId, String regNo, String name,
                    int route, double amount) throws IOException {

        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE,
                StandardOpenOption.APPEND)) {

            w.write(String.join(",", passId, regNo, name,
                    String.valueOf(route),
```

```
                String.valueOf(amount)));

        w.newLine();
    }
}

public String[] find(String passId) throws IOException {

    try (BufferedReader br = Files.newBufferedReader(file)) {

        String line;

        while ((line = br.readLine()) != null) {

            String[] fields = line.split(",", -1);

            if (fields.length != 5) {
                continue;
            }

            if (fields[0].equals(passId)) {
                return fields;
```

```
            }
        }
    }

    return null;
}

public static void main(String[] args) throws IOException {

    Path f = Files.createTempFile("passes", ".csv");

    PassStore store = new PassStore(f);

    store.add("P1001", "192311001", "Arun", 12, 4500);

    Files.writeString(f, "P1002,DAMAGED LINE\n",
            StandardOpenOption.APPEND);

    store.add("P1003", "192311003", "Divya", 7, 3800);

    System.out.println(Arrays.toString(store.find("P1003")));
    System.out.println(Arrays.toString(store.find("P1002")));
```

```
        Files.delete(f);
    }
}
```

Expected output:

[P1003, 192311003, Divya, 7, 3800.0]

null

Explanation

- BufferedReader reads the file one line at a time.
- split(",", -1) separates the fields and preserves empty fields.
- fields.length != 5 identifies a damaged record.
- continue skips the damaged record.
- If the Pass ID matches, the record is returned.
- If no record matches, the method returns null.

Why does the damaged P1002 line not stop the search?

The damaged line contains only two fields instead of five. The condition fields.length != 5 becomes true, so continue skips that line and moves to the next record. Therefore, the program can still find P1003.

B1.2 – Validate the Implementation

Test case	Input	Expected result	Purpose
Normal case	P1003	Divya's record	Checks successful search
Missing ID	P9999	null	Checks unavailable record

Damaged line	P1002	null	Checks damaged-record handling

B1.3 – Measure the Effect of Buffering

Measure the time taken to count lines in files of 1 MB, 10 MB and 50 MB.

Use this table to enter your actual results after executing both versions.

File size	Unbuffered time (ms)	Buffered time (ms)
1 MB	_____	_____
10 MB	_____	_____
50 MB	_____	_____

Expected observation:

Buffered reading should generally be faster because it reduces repeated input operations. Actual timings depend on the computer, storage device, buffer size and file contents.

B1.4 – Measure Growth

Measure the time to find a pass as the file grows.

Number of records	Search time (ms)
1,000	_____
10,000	_____
50,000	_____
100,000	_____

Interpretation:

- The program searches records sequentially.
- As the number of records increases, the search may take more time.
- The worst-case time complexity is O(n).

- Additional memory usage is O(1) for the line-by-line search, excluding the current record and reader buffer.

Alternative Concept – HashMap

Instead of searching the file every time, all records can be loaded into a HashMap, using Pass ID as the key.

Example:

```
import java.util.*;


public class PassMapDemo {


   public static void main(String[] args) {


      HashMap<String, String[]> passes = new HashMap<>();


      passes.put("P1001",

         new String[]{"P1001", "192311001", "Arun", "12", "4500"});


      passes.put("P1003",

         new String[]{"P1003", "192311003", "Divya", "7", "3800"});


      System.out.println(Arrays.toString(passes.get("P1003")));

   }

}
```

Complexity comparison:

Method	Loading time	Search time	Extra memory
File scanning	No initial loading	O(n)	O(1)
HashMap	O(n)	Average O(1)	O(n)

For q searches on n records:

- File scanning: O(q × n).
- HashMap: O(n + q) on average.

TASK B2 – ENGINEER AN ADVERSARIAL INPUT **5 Marks**

Bloom's Level: K4 – Analyse

B2.1 – Unsafe Login Check

The unsafe login method directly combines user input with SQL statements.

Unsafe SQL:

```
String sql = "SELECT * FROM users WHERE id='" +
        id + "' AND pwd='" + pwd + "'";
```

Adversarial input:

User ID: admin' OR '1'='1' --

Password: test

This input attempts to modify the SQL condition and bypass password validation.

B2.2 – PreparedStatement Version

```
String sql = "SELECT * FROM users WHERE id=? AND pwd=?";


PreparedStatement ps = con.prepareStatement(sql);


ps.setString(1, id);
```

```
ps.setString(2, pwd);


ResultSet rs = ps.executeQuery();


if (rs.next()) {
   System.out.println("Login Successful");
} else {
   System.out.println("Invalid Login");
}
```

The PreparedStatement treats the input as parameter values rather than executable SQL.

B2.3 – Damaged Bus-Pass File

Create a test file named damaged.csv.

P1001,192311001,Arun,12,4500

P1002,DAMAGED LINE

P1003,192311003,Divya,7,ABC

P1004,192311004,Ravi,5,

Test cases:

Record	Problem	Expected handling
P1001	Valid record	Accept
P1002	Wrong number of fields	Skip
P1003	Non-numeric amount	Reject during numeric validation
P1004	Empty amount	Reject during numeric validation

B2.4 – Explanation

- A wrong number of fields indicates an incomplete record.
- A non-numeric amount cannot be converted into a valid numeric value.
- An empty amount means required information is missing.
- Input validation prevents damaged data from entering the application.

Alternative Concept – Escaping or Regular Expressions

Escaping quotation marks or validating user input with regular expressions is weaker than using PreparedStatement.

Reasons:

1. Manual escaping can miss special SQL syntax.
2. Different databases may handle escaping differently.
3. New attack patterns may not be covered by a regular expression.
4. PreparedStatement separates SQL commands from user input.

Complexity: Validation generally takes time proportional to the input length, O(m), where m is the input length. PreparedStatement does not eliminate database query cost, but it provides a safer parameter-handling mechanism.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH **8 Marks**

Bloom's Level: K4 – Analyse

Move the bus-pass records into a MySQL table using JDBC.

B3.1 – Create the Bus-Pass Table

Create a database named transport_db in MySQL.

SQL code:

```
CREATE DATABASE transport_db;


USE transport_db;


CREATE TABLE bus_pass (
    pass_id VARCHAR(20) PRIMARY KEY,
```

```
    reg_no VARCHAR(20),
    name VARCHAR(100),
    route_no INT,
    amount DOUBLE
);
```

Insert 10 Records

Using Statement:

```
Statement st = con.createStatement();


st.executeUpdate(
    "INSERT INTO bus_pass VALUES " +
    "('P1001','192311001','Arun',12,4500)"
);
```

Using PreparedStatement:

```
String sql = "INSERT INTO bus_pass VALUES (?, ?, ?, ?, ?)";


PreparedStatement ps = con.prepareStatement(sql);


ps.setString(1, "P1002");
ps.setString(2, "192311002");
ps.setString(3, "Divya");
ps.setInt(4, 7);
```

```
ps.setDouble(5, 3800);


ps.executeUpdate();
```

Using SELECT to verify records:

```
Statement st = con.createStatement();


ResultSet rs = st.executeQuery("SELECT * FROM bus_pass");


while (rs.next()) {
    System.out.println(
        rs.getString("pass_id") + " " +
        rs.getString("name") + " " +
        rs.getDouble("amount")
    );
}
```

B3.2 – Verify Column Types Using ResultSetMetaData

```
ResultSetMetaData md = rs.getMetaData();


int count = md.getColumnCount();


for (int i = 1; i <= count; i++) {
```

```
    System.out.println(
        md.getColumnName(i) + " - " +
        md.getColumnTypeName(i)
    );
}
```

Expected column information:

Column	Data type
pass_id	VARCHAR
reg_no	VARCHAR
name	VARCHAR
route_no	INT
amount	DOUBLE

B3.3 – Complete the Code (TODO 1, TODO 2 and TODO 3)

The following program inserts bus-pass records in batches of 1,000 rows using a single transaction.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
```

```
public static int loadAll(Connection con, List<String[]> passes)
    throws SQLException {

  String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) "
          + "VALUES (?, ?, ?, ?, ?)";

  boolean oldAutoCommit = con.getAutoCommit();

  con.setAutoCommit(false);

  try (PreparedStatement ps = con.prepareStatement(sql)) {

    int n = 0;

    for (String[] p : passes) {

      // TODO 1
      ps.setString(1, p[0]);
      ps.setString(2, p[1]);
      ps.setString(3, p[2]);
      ps.setInt(4, Integer.parseInt(p[3]));
```

```
            ps.setDouble(5, Double.parseDouble(p[4]));


            ps.addBatch();
            n++;


            if (n % 1000 == 0) {
                ps.executeBatch();
            }
        }

        // Execute remaining records
        ps.executeBatch();


        // TODO 2
        con.commit();
        return n;

    } catch (SQLException | RuntimeException e) {


        // TODO 3
        con.rollback();
        throw e;
```

```
        } finally {


            con.setAutoCommit(oldAutoCommit);

        }

    }

}
```

Explanation of completed TODOs:

- TODO 1: Sets the five SQL parameters, adds each record to the batch and executes every 1,000 records.
- TODO 2: Commits the transaction and returns the number of records inserted.
- TODO 3: Rolls back the transaction if a database or runtime error occurs, then rethrows the exception.

B3.4 – Test the Program

Call loadAll() with 1,000, 10,000 and 100,000 generated records.

Example call:

```
int count = PassLoader.loadAll(con, passes);



System.out.println("Records Loaded: " + count);
```

Expected output for a successful 1,000-record test:

Records Loaded: 1000

This is an expected result, not a measured execution result. Run the program to obtain your actual output and timings.

B3.5 – Auto-commit vs Transaction

Feature	Auto-commit ON	Single transaction
Saving	Each statement is committed separately	All records are committed together
Failure halfway	Earlier successful inserts may remain	All inserts can be rolled back
Reliability	Lower for large imports	Higher
Control	Limited	Full commit and rollback control

Measurement table:

Records	Statement (ms)	PreparedStatement (ms)	Batch (ms)
1,000	_____	_____	_____
10,000	_____	_____	_____
100,000	_____	_____	_____

Alternative Concept – Multi-row INSERT and LOAD DATA

1. Multi-row INSERT

Multiple records can be inserted using one SQL statement.

INSERT INTO bus_pass

(pass_id, reg_no, name, route_no, amount)

VALUES

('P1001','192311001','Arun',12,4500),

('P1002','192311002','Divya',7,3800);

2. LOAD DATA

MySQL can load many records directly from a CSV file.

```
LOAD DATA LOCAL INFILE 'passes.csv'
INTO TABLE bus_pass
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
(pass_id, reg_no, name, route_no, amount);
```

Comparison:

Method	**Advantage**	**Limitation**
PreparedStatement batch	Safe parameter handling and transaction support	Requires batch management
Multi-row INSERT	Fewer SQL executions	Query size can become large
LOAD DATA	Designed for bulk loading	Requires suitable file access and database configuration

For n records, all approaches must process the input data. A batch of b records uses memory proportional to O(b) for the batch data, apart from other program memory. Actual performance should be established through measurements.

TASK B4 – FINAL ENGINEERING DECISION **5 Marks**

Bloom's Level: K5 – Evaluate

The transport office will now serve all 12 campuses of the university, with 100× the current number of passes and several counters issuing passes at the same time.

B4.1 – Implement a Practical Strategy

The transport office needs to migrate existing bus-pass records from a text file into a database.

Migration process:

1. Read the existing CSV file using BufferedReader.
2. Read each record one by one.
3. Validate the required fields.
4. Store valid records in batches.
5. Insert the batches into MySQL.
6. Commit successful records as a transaction.
7. Record invalid records in a separate failure report.

Pseudocode:

```
BEGIN
   OPEN input CSV file
   CONNECT TO DATABASE

   DISABLE AUTO COMMIT

   INITIALIZE validRecords
   INITIALIZE failedRecords

   WHILE another line is available
      READ line
      SPLIT line into fields

      IF record is valid THEN
         ADD record to validRecords

         IF batch size reaches 1000 THEN
```

```
            INSERT batch into database
            CLEAR batch
        END IF

    ELSE
        ADD record to failedRecords
    END IF
END WHILE

INSERT remaining valid records
COMMIT TRANSACTION

SAVE failedRecords into error report

CLOSE file and database connection

DISPLAY migration summary
END
```

Example migration summary format:

Migration Completed

Total Records Read: 10000

Valid Records: 9980

Failed Records: 20

Records Inserted: 9980

Illustrative format only. Replace the numbers with your actual program results.

B4.2 – Inspect the Database Using DatabaseMetaData

DatabaseMetaData provides information about the database and its JDBC driver.

Java code:

```
import java.sql.*;


public class DatabaseInfo {


    public static void main(String[] args) throws Exception {


        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/transport_db",
            "root",
            "password"
        );


        DatabaseMetaData md = con.getMetaData();


        System.out.println(
            "Database: " + md.getDatabaseProductName()
        );


        System.out.println(
            "Driver Version: " + md.getDriverVersion()
        );
```

```
        ResultSet rs = md.getTables(
            null, null, "%", new String[]{"TABLE"}
        );

        System.out.println("Tables:");

        while (rs.next()) {
            System.out.println(rs.getString("TABLE_NAME"));
        }

        rs.close();
        con.close();
    }
}
```

Expected output format:

Database: MySQL

Driver Version: [Installed driver version]

Tables:

bus_pass

The exact driver version depends on the JDBC driver installed on the computer.

Importance of DatabaseMetaData:

- Identifies the database product.
- Displays the JDBC driver version.
- Lists available database tables.

- Helps check database compatibility.
- Helps when moving the application to another server.

B4.3 – Production Decision

The transport office must select a suitable storage method for 12 campuses and 100 times the current number of passes.

Factor	**Text File**	**Serialization**	**Database**
Data storage	Text records	Java objects	Structured tables
Searching	Requires scanning	Requires object loading or indexing	SQL queries and indexes
Multiple users	Difficult to coordinate	Difficult to coordinate	Supports concurrent access
Data security	Depends on file permissions	Depends on file permissions	Access controls and permissions
Large data	Less convenient	Less convenient for shared access	Suitable
Transactions	Must be implemented separately	Must be implemented separately	Supports transactions

Recommended solution: Relational Database (MySQL)

Reasons:

1. It supports multiple counters accessing records at the same time.
2. SQL makes searching and filtering bus-pass records easier.
3. Transactions protect records from incomplete updates.
4. PreparedStatement helps prevent SQL injection.
5. Database indexing can improve search performance as the number of records grows.
6. It supports centralized management across all 12 campuses.

Conclusion:

A relational database is the most suitable solution for the expanded transport office because it provides centralized storage, concurrent access, efficient searching and transaction support.

Alternative Concept – Separate Serialized Files for Each Campus

If every campus maintains its own serialized file:

- Each campus can store its own bus-pass records.
- Data must be combined when a university-wide search is required.
- Each campus must manage its own backup and recovery.
- Concurrent access to the same file must be controlled.
- A centralized database would be easier to manage across all campuses.

Complexity comparison:

Operation	**Separate serialized files**	**Shared indexed database**
Search one campus	O(n) without an index	Depends on the database index
Search all 12 campuses	O(12n) if each file is scanned	Can use one query
Concurrent counters	Requires file-level coordination	Database transaction and locking support
Data sharing	Requires file exchange or synchronization	Centralized access

MARK DISTRIBUTION

Component	Task	Bloom's Level	Marks
Part A – Design	A1 – File Format Design	K4 – Analyse	7
	A2 – Buffered I/O Design	K4 – Analyse	5
	A3 – Data-Access Pattern	K4 – Analyse	5
	A4 – Serialization Design	K4 – Analyse	4
	A5 – Transactions & Injection	K4 – Analyse	4
	Part A Total		**25**
Part B – Implementation	B1 – Instrumented File Store	K4 – Analyse	7
	B2 – Adversarial Inputs	K4 – Analyse	5
	B3 – Statement vs PreparedStatement vs Batch	K4 – Analyse	8
	B4 – Final Engineering Decision	K5 – Evaluate	5
	Part B Total		**25**
	TOTAL		**50**